

CPP - Module 02

Forma Canónica Ortodoxa, punto fijo y sobrecarga de operadores

De cero a entenderlo todo de verdad

INDICE

PARTE 1 — La nueva regla: la Forma Canónica Ortodoxa

1. Qué cambia en CPP02 y por qué
2. El problema sin OCF — la copia superficial
3. Las 4 funciones obligatorias — el mapa completo
4. El constructor por defecto
5. El constructor de copia — en profundidad
6. El operador de asignación por copia — en profundidad
7. Constructor de copia VS operador de asignación — la diferencia definitiva
8. El destructor
9. La Regla de los Tres
10. `return *this` — por qué y para qué
11. La comprobación de autoasignación — `if (this != &other)`
12. La lista de inicialización — por qué es más que un estilo
13. `static const int` — la constante de clase

PARTE 2 — Los números de punto fijo — el concepto desde cero

14. El problema con los enteros
15. El problema con los flotantes — la imprecisión que duele
16. La solución — punto fijo
17. La representación binaria con cajones
18. El factor de escala — por qué 256
19. El

epsilon — el valor mínimo representable 20. La precisión tiene límites — qué no puedes representar

PARTE 3 — Convertir hacia y desde punto fijo 21. De int a punto fijo — el constructor int 22. El desplazamiento de bits — << y >> explicados desde cero 23. De float a punto fijo — el constructor float 24. roundf — por qué redondear y no truncar 25. roundf vs floorf vs ceilf — las diferencias 26. De punto fijo a float — toFloat 27. El cast en C++ — (float), (int) 28. Por qué sin el cast la división trunca 29. De punto fijo a int — toInt

PARTE 4 — Qué es la sobrecarga de operadores 30. Ad-hoc polymorphism — el mismo símbolo, comportamientos distintos 31. Cómo declara el compilador un operador sobrecargado 32. Operador como método vs operador como función libre

PARTE 5 — Los operadores de comparación 33. La firma de los operadores de comparación 34. Por qué operar sobre _value y no sobre toFloat 35. El if con return — las tres formas equivalentes

PARTE 6 — Los operadores aritméticos 36. La firma — por qué devuelven Fixed por valor 37. El objeto temporal — qué es y cuándo nace y muere 38. Aritmética con toFloat vs aritmética directa sobre _value

PARTE 7 — Los operadores de incremento y decremento 39. Pre-incremento ++a — firma, comportamiento, traza en RAM 40. Post-incremento a++ — firma, comportamiento, traza en RAM 41. El parámetro int dummy — por qué existe 42. Pre-decremento --a 43. Post-decremento a-- 44. El epsilon en contexto — qué significa _value += 1

PARTE 8 — El operador << de inserción 45. Por qué operator<< no puede ser método de la clase 46. La firma completa desglosada símbolo por símbolo 47. El encadenamiento — por qué devuelve ostream&

PARTE 9 — Las funciones estáticas min y max 48. Por qué son static — y qué significa static en un método 49. La propagación de const — por qué hay dos versiones 50. Devolver referencia — por qué no por valor 51. Cómo llamarlas desde el main

PARTE 10 — Ex03: BSP y la clase Point 52. Qué pide el ex03 53. La clase Point en OCF 54. El problema de los atributos const — Fixed const x 55. El operator= con atributos const — la trampa 56. El algoritmo BSP — cómo saber si un punto está en un triángulo 57. El producto vectorial — el corazón del algoritmo 58. Implementación completa del bsp

PARTE 11 — Los ejercicios explicados uno a uno 59. Ex00: My First Class in Orthodox Canonical Form 60. Ex01: Towards a more useful fixed-point number class 61. Ex02: Now we're talking 62. Ex03: BSP (bonus)

Glosario completo

PARTE 1 — LA FORMA CANÓNICA ORTODOXA

1. Qué cambia en CPP02 y por qué

El subject lo dice sin rodeos en el Capítulo III:

“From now on, all your classes must be designed in the Orthodox Canonical Form.”

Esto aplica desde CPP02 hasta CPP09 sin excepción. Cada clase que crees — sin importar lo simple que sea — debe tener obligatoriamente cuatro funciones concretas.

Antes de CPP02 podías definir solo los constructores que necesitabas. A partir de ahora, si cualquiera de esas cuatro funciones falta, tu código no cumple la norma del subject.

Las cuatro funciones son:

- | | |
|----------------------------|---|
| 1. Constructor por defecto | → <code>Fixed()</code> |
| 2. Constructor de copia | → <code>Fixed(const Fixed &other)</code> |
| 3. Operador de asignación | → <code>Fixed &operator=(const Fixed &other)</code> |
| 4. Destructor | → <code>~Fixed()</code> |

Para entender por qué existen estas cuatro, necesitas entender primero el problema que resuelven.

2. El problema sin OCF — la copia superficial

Cuando no defines estas funciones, el compilador genera versiones automáticas. Esas versiones hacen lo que se llama una **copia superficial** (shallow copy) — copian los valores de los atributos byte por byte, sin pensar en lo que significan.

Para una clase con solo un `int` esto funciona. Pero imagina una clase con un puntero a memoria dinámica:

```
class Imagen {  
    int _ancho;  
    int _alto;  
    int *_pixeles;    // puntero a un array en el heap  
};
```

Sin definir las funciones, el compilador copia el puntero — no lo que apunta:

STACK: HEAP:

img1._pixeles = 0x7f3a $\longrightarrow$ [pixel0][pixel1][pixel2]...[pixelN]

STACK: HEAP:

img1._píxeles = 0x7f3a → [pixel0][pixel1][pixel2]...[pixelN]

img2._píxeles = 0x7f3a —————┐

 misma dirección – dos objetos apuntan al mismo bloque

```
~Imagen() { delete[] _píxeles; }
```

La OCF te obliga a definir explícitamente qué pasa en cada situación. Para `Fixed` con su simple `int _value`, la copia superficial funcionaría — pero la OCF es la norma de todas formas y debes seguirla.

Antes de entrar en cada una, aquí el mapa de cuándo se llama cada función:

La regla que hay que memorizar:

`Fixed c = a` parece una asignación por el `=`, pero `c` no existía antes — es un constructor de copia.

4. El constructor por defecto

```
// declaración en Fixed.hpp
Fixed();

// implementación en Fixed.cpp
Fixed::Fixed() : _fPvalue(0) {
    std::cout << "Default constructor called" << std::endl;
}
```

Se llama cuando creas un objeto **sin parámetros**:

```
Fixed a;           // una llamada al constructor por defecto
Fixed array[8];    // ocho llamadas al constructor por defecto – una por elemento
```

Su trabajo: dejar el objeto en un estado válido desde el primer instante. Para `Fixed`, eso significa `_fPvalue = 0`, que en punto fijo representa el valor 0.0.

La parte `: _fPvalue(0)` es la **lista de inicialización**. La explicamos en profundidad en la sección 12. Por ahora: inicializa el atributo directamente con el valor correcto, antes de ejecutar el cuerpo `{ }`.

El `Fixed::` antes del nombre es el **operador de resolución de ámbito** — le dice al compilador “este constructor pertenece a la clase `Fixed`”. En el `.hpp` no lo necesitas. En el `.cpp` es obligatorio para todas las implementaciones de métodos de clase.

5. El constructor de copia — en profundidad

```
// declaración en Fixed.hpp
Fixed(const Fixed &other);

// implementación en Fixed.cpp
Fixed::Fixed(const Fixed &other) : _fPvalue(0) {
    std::cout << "Copy constructor called" << std::endl;
    *this = other;
}
```

Se llama cuando creas un objeto nuevo a partir de uno que ya existe:

```
Fixed a;
Fixed b(a);    // constructor de copia – b se CREA a partir de a
```

```
Fixed c = a;    // también constructor de copia — c no existía
```

La firma — por qué `const` y por qué referencia

```
Fixed(const Fixed &other);  
//      ^^^^^ ^  
//      const  referencia — AMBOS OBLIGATORIOS
```

Por qué referencia (`&`):

Si no fuera referencia, el parámetro se pasaría **por valor**. Pasar por valor significa hacer una copia del argumento antes de dárselo a la función. Para hacer esa copia... el compilador llamaría al constructor de copia. Que para copiar su argumento... llamaría al constructor de copia. Bucle infinito. El compilador lo detecta en tiempo de compilación y da error.

La referencia rompe el bucle — no hay copia, se pasa directamente la dirección del objeto original.

Por qué `const` :

Prometes que no vas a modificar el objeto del que copias. Además, `const` es necesario para que puedas copiar objetos que son constantes:

```
const Fixed x(5);  
Fixed y(x);    // sin const en la firma → ERROR  
               // con const en la firma → OK
```

El cuerpo — por qué `*this = other`

```
Fixed::Fixed(const Fixed &other) : _fPvalue(0) {  
    std::cout << "Copy constructor called" << std::endl;  
    *this = other;    // llama al operador de asignación  
}
```

`*this = other` llama al operador de asignación que ya implementaste. Así la lógica de copia vive en un solo sitio. Si mañana la clase crece y el operador de asignación cambia, el constructor de copia se actualiza automáticamente.

El `: _fPvalue(0)` en la lista de inicialización es necesario para que el objeto tenga un valor inicial antes de ejecutar `*this = other`. Sin él, `_fPvalue` contendría basura hasta que el operador de asignación lo sobrescriba — en C++ los enteros no se inicializan solos.

Trazando en RAM con `Fixed b(a)` donde `a._fPvalue = 0`:

```
b se construye:
Lista de inicialización: b._fPvalue = 0
imprime "Copy constructor called"
*this = other → llama a b.operator=(a)
imprime "Copy assignment operator called"
this != &other? b != a? Sí
b._fPvalue = a.getRawBits() → b._fPvalue = 0
return *this → return b
b queda con _fPvalue = 0 
```

6. El operador de asignación por copia — en profundidad

```
// declaración en Fixed.hpp
Fixed &operator=(const Fixed &other);

// implementación en Fixed.cpp
Fixed &Fixed::operator=(const Fixed &other) {
    std::cout << "Copy assignment operator called" << std::endl;
    if (this != &other)
        this->_fPvalue = other.getRawBits();
    return *this;
}
```

Se llama cuando asignas a un objeto que **ya existe**:

```
Fixed a;    // existe
Fixed b;    // existe
b = a;      // operador de asignación — b ya existía
```

La firma — por qué devuelve Fixed&

```
Fixed &operator=(const Fixed &other);
// ^^^
// devuelve referencia
```

Devolver `Fixed &` en lugar de `void` permite el encadenamiento de asignaciones:

```
Fixed a, b, c;
a = b = c;
// El compilador lo procesa de derecha a izquierda:
```

```
// Paso 1: b = c → b.operator=(c) → devuelve referencia a b
// Paso 2: a = b → a.operator=(b) → devuelve referencia a a
```

Si devolviera `void`, el paso 2 no tendría operando derecho y el compilador daría error.

this — el puntero oculto

`this` es un puntero especial que existe dentro de cualquier método de clase. Apunta al objeto sobre el que se está ejecutando el método.

```
Fixed a, b;
b = a;
// dentro del operator=:
//   this = &b   (puntero a b — el objeto que se está modificando)
//   other = a   (referencia a a — el objeto del que se copia)
```

En RAM:

```
STACK:
a → cajon 1040: [ _fPvalue = 256 ]
b → cajon 2080: [ _fPvalue = 0   ]
```

```
dentro de b.operator=(a):
    this = 2080   (dirección de b)
    &other = 1040 (dirección de a)
```

La comprobación de autoasignación

```
if (this != &other)
    this->_fPvalue = other.getRawBits();
```

`this` es la dirección del objeto actual. `&other` es la dirección del objeto que se pasa. Si son iguales, alguien hizo `a = a` — autoasignación. No tiene sentido copiar nada.

Para `Fixed` con un simple `int`, la autoasignación no causa crash. Pero en clases con punteros es catastrófica:

```
// clase con puntero — SIN comprobación:
Fixed &operator=(const Fixed &other) {
    delete[] _datos;                // borra los datos actuales
    _datos = new int[other._tamano]; // reserva espacio
    copiar(other._datos, _datos);    // copia los datos de other
    // si this == &other: acabas de borrar los datos que ibas a copiar
```



```
}
```

return *this

```
return *this;
```

`this` es el puntero al objeto actual (tipo `Fixed *`). `*this` es el objeto en sí (tipo `Fixed`). Devolver `*this` devuelve el objeto actual como referencia (`Fixed &`) — lo que el tipo de retorno promete.

Trazando `a = b = c`:

```
Paso 1: b = c
  b.operator=(c) se ejecuta
  b._fPvalue = c._fPvalue
  return *this → devuelve referencia a b
```

```
Paso 2: a = (referencia a b)
  a.operator=(b) se ejecuta
  a._fPvalue = b._fPvalue
  return *this → devuelve referencia a a
```

7. Constructor de copia VS operador de asignación — la diferencia definitiva

Esta es la confusión más frecuente en CPP02. La tabla:

	Constructor de copia	Operador de asignación
El destino	NO existe todavía	YA existe
Firma	<code>Fixed(const Fixed &)</code>	<code>Fixed &operator=(const Fixed &)</code>
Tipo de retorno	Ninguno (es constructor)	<code>Fixed &</code>
Comprobación <code>this</code>	No necesaria	Sí — <code>if (this != &other)</code>
Sintaxis	<code>Fixed b(a)</code> o <code>Fixed b = a</code>	<code>b = a</code> (b ya existía)

```
Fixed a;           // constructor por defecto
```

```
Fixed b(a);      // constructor de COPIA — b no existía
Fixed c = a;     // constructor de COPIA — c no existía (pese al =)
Fixed d;         // constructor por defecto — d se crea
d = a;          // operador de ASIGNACIÓN — d ya existía
```

8. El destructor

```
// declaración en Fixed.hpp
~Fixed();

// implementación en Fixed.cpp
Fixed::~~Fixed() {
    std::cout << "Destructor called" << std::endl;
}
```

El destructor se llama **automáticamente** cuando el objeto va a ser destruido. Hay dos casos:

Caso 1 — al salir del scope:

```
{
    Fixed a;    // constructor llamado
    // ...
}              // destructor de a llamado aquí, automáticamente
```

Caso 2 — al hacer delete:

```
Fixed *p = new Fixed();
delete p;    // destructor llamado aquí, tú decides cuándo
```

Para `Fixed` el destructor no tiene trabajo especial — `_fpvalue` es un `int` del stack, se destruye solo. Pero debe existir, declararse e implementarse — la OCF lo exige.

El orden de destrucción — LIFO

Los objetos del stack se destruyen en orden **inverso** al de creación:

```
Fixed a;    // creado primero
Fixed b;    // creado segundo
Fixed c;    // creado tercero

// al salir del scope:
```

```
// → destructor de c (último en crearse, primero en destruirse)
// → destructor de b
// → destructor de a (primero en crearse, último en destruirse)
```

Por eso el output del ex00 termina con tres "Destructor called" — uno por cada objeto, en orden inverso a como aparecieron.

9. La Regla de los Tres

Existe una regla en C++ llamada **Rule of Three**:

Si tu clase necesita definir cualquiera de estas tres funciones, probablemente necesita las tres:

- Destructor personalizado
- Constructor de copia personalizado
- Operador de asignación personalizado

Por qué: si necesitas un destructor personalizado es porque gestionas un recurso (memoria, fichero...). Si gestionas un recurso, la copia automática del compilador (shallow copy) será incorrecta. Si la shallow copy es incorrecta, necesitas las otras dos.

La OCF de 42 añade el constructor por defecto — haciéndola la **Regla de los Cuatro** en la práctica.

En C++11 esta regla se extendió a la **Regla de los Cinco** añadiendo move constructor y move assignment operator. En C++98 solo aplica la de tres.

10. return *this — por qué y para qué

Esto merece su propia sección porque genera confusión.

```
Fixed &Fixed::operator=(const Fixed &other) {
    if (this != &other)
        _fPvalue = other._fPvalue;
    return *this;    // ← esto
}
```

Desglosado:

`this` → puntero al objeto actual (tipo: `Fixed *`)
contiene la dirección del objeto en RAM, p.ej. `0x7fff2080`

`*this` → el objeto al que apunta `this` (tipo: `Fixed`)
es el objeto en sí, no un puntero

`return *this` → devuelve el objeto como `Fixed &`
(referencia al objeto actual – no una copia)

En RAM:

STACK durante `operator=`:

`this = 0x7fff2080` ← dirección de `b` (el objeto que recibe la asignación)

`return *this` → devuelve referencia al objeto en `0x7fff2080`
→ quien llame a `operator=` puede usar `b` directamente

Para qué sirve — el encadenamiento:

```
a = b = c;  
// sin return *this:  
//   b = c → devuelve void → nada  
//   a = (nada) → ERROR  
// con return *this:  
//   b = c → devuelve referencia a b  
//   a = b → funciona
```

11. La comprobación de autoasignación — `if (this != &other)`

```
if (this != &other)  
    _fPvalue = other._fPvalue;
```

Comparamos dos **punteros** — no dos objetos:

`this` → dirección del objeto actual
`&other` → dirección del objeto que se pasa como parámetro

`if (this != &other)` → si son direcciones distintas → no es autoasignación

Si `this == &other`, el objeto origen y el destino son **el mismo cajón en RAM**. Ejecutar la

asignación no tiene sentido — y en clases con punteros sería catastrófico.

Para `Fixed`:

```
Fixed a(5.0f);
a = a;    // autoasignación

// sin comprobación: _fPvalue = a._fPvalue → ok para int, no modifica nada
// con comprobación: detecta que this == &a → se salta la asignación
```

Para una clase con punteros:

```
// SIN comprobación — clase con puntero a array:
a = a;
delete[] _datos;          // BORRA los propios datos
_datos = new int[...];    // reserva nuevo espacio
copiar(a._datos, _datos); // intenta copiar los datos que acabas de borrar
// comportamiento indefinido garantizado
```

12. La lista de inicialización — por qué es más que un estilo

La lista de inicialización es la parte que va entre `:` y `{` en un constructor:

```
Fixed::Fixed() : _fPvalue(0) {
//              ^^^^^^^^^^^^^^
//              lista de inicialización
}
```

Puede parecer solo una cuestión de estilo respecto a asignar en el cuerpo. No lo es.

Qué hace exactamente

La lista de inicialización **construye el atributo directamente con el valor correcto**. El cuerpo del constructor se ejecuta después — sobre atributos ya contruidos.

Sin lista de inicialización:

```
Fixed::Fixed() {
    _fPvalue = 0;    // el compilador primero inicializa _fPvalue con basura,
                    // luego sobrescribe con 0
                    // dos operaciones donde bastaría una
}
```

Con lista de inicialización:

```
Fixed::Fixed() : _fPvalue(0) {  
    // _fPvalue ya nació con 0 – una sola operación  
}
```

Cuándo es OBLIGATORIA y no solo recomendada

La lista de inicialización es **obligatoria** en tres casos:

Caso 1 — atributos que son referencias:

```
class HumanA {  
    Weapon &_weapon;    // referencia – segunda etiqueta de otro cajón  
};  
  
HumanA::HumanA(Weapon &w) : _weapon(w) { }    // OBLIGATORIO  
// HumanA::HumanA(Weapon &w) { _weapon = w; } // ERROR – no puedes asignar una re
```

Una referencia es una segunda etiqueta — debe existir desde el momento en que el objeto nace. No puedes construir el objeto con la referencia “vacía” y asignarla después.

Caso 2 — atributos `const`:

```
class Point {  
    const Fixed _x;    // constante – no puede modificarse tras la construcción  
};  
  
Point::Point(float x) : _x(x) { }    // OBLIGATORIO  
// Point::Point(float x) { _x = x; } // ERROR – no puedes asignar a un const
```

Un atributo `const` solo puede tener su valor en el momento de construcción. Una vez construido, es inmutable. Esto tiene implicaciones importantes para el ex03 — ver sección 54.

Caso 3 — clases sin constructor por defecto como atributos:

Si tu clase tiene un atributo de otro tipo que no tiene constructor por defecto, debes inicializarlo en la lista.

13. static const int — la constante de clase

```
class Fixed {
    private:
        int          _fPvalue;
        static const int _bits = 8;    // ← esto
};
```

Dos modificadores combinados:

static — pertenece a la clase, no al objeto

Un atributo normal existe en cada objeto por separado:

```
Fixed a, b, c;

STACK:
a → cajon 1040: [ _fPvalue = 0 ]
b → cajon 2080: [ _fPvalue = 256 ]
c → cajon 3120: [ _fPvalue = 512 ]
```

Un atributo `static` existe una sola vez para toda la clase:

```
Fixed a, b, c;

STACK:
a → cajon 1040: [ _fPvalue = 0   ]
b → cajon 2080: [ _fPvalue = 256 ]
c → cajon 3120: [ _fPvalue = 512 ]

DATOS GLOBALES:
Fixed::_bits = 8    ← uno solo, compartido por a, b y c
```

const — nunca cambia

El número de bits fraccionarios es siempre 8. Marcarla `const` hace que el compilador lo garantice. Si escribes `_bits = 16` en cualquier parte del código, error de compilación inmediato.

Inicialización en el header — solo para enteros

Para tipos enteros (`int`, `short`, `long` ...) con `static const`, C++98 permite inicializar directamente en el header:

```
static const int _bits = 8;    // permitido solo para enteros en C++98
```

Para otros tipos (`float` , `std::string` ...) la inicialización debe ir en el `.cpp` fuera de la clase. Esta es una excepción especial de C++98.

PARTE 2 — LOS NÚMEROS DE PUNTO FIJO

14. El problema con los enteros

Los enteros (`int` , `long` ...) solo representan números sin parte decimal: ..., -2, -1, 0, 1, 2, ...

No pueden representar 3.14, 0.5, -7.25. Si necesitas decimales con enteros, estás atrapado.

15. El problema con los flotantes — la imprecisión que duele

Los flotantes (`float` , `double`) sí representan decimales. Pero con un problema fundamental: **no son exactos**.

Los números en RAM se representan en binario (base 2). La mayoría de decimales en base 10 no tienen representación exacta en base 2 — igual que 1/3 no tiene representación exacta en base 10 (0.333333...).

```
float f = 0.1f + 0.2f;
// En teoría: 0.3
// En RAM:    0.30000001192093...
```

Trazando en binario:

```
0.1 en binario: 0.00011001100110011001100110011... (infinito)
0.2 en binario: 0.00110011001100110011001100110... (infinito)
```

```
El float tiene 32 bits — tiene que truncar en algún sitio
El error de truncación se acumula con cada operación
```

Este error es inaceptable en:

- **Gráficos 3D** — si los vértices no cuadran exactamente, aparecen artefactos visuales
- **Procesado de audio** — errores de precisión generan ruido audible
- **Simulaciones físicas** — el error se acumula y la simulación se vuelve incorrecta

- **Sistemas financieros** — $0.1 + 0.2 \neq 0.3$ es un bug, no una curiosidad

16. La solución — punto fijo

Los números de punto fijo representan decimales usando **solo enteros** — pero con una convención acordada sobre dónde está el “punto decimal”.

La idea: si acordamos que los últimos 8 bits del entero representan la parte fraccionaria, podemos representar decimales usando aritmética de enteros — exacta y rápida.

Cuando dices `_bits = 8`, estás diciendo:

“El valor real de este número = `_fPvalue` dividido entre 2^8 ”

Todo lo demás se deriva de eso.

17. La representación binaria con cajones

Un `int` en C++ ocupa 32 bits — 32 cajoncitos de 0 o 1. Con `_bits = 8`, los dividimos así:

```
bit 31                bit 8  bit 7                bit 0
|                    |   |                    |
[ parte entera - 24 bits ] [ parte decimal - 8 bits ]
```

Los 8 bits más bajos (bits 0-7) son la parte fraccionaria. Los 24 bits más altos (bits 8-31) son la parte entera.

Ejemplo con el número 42.5:

```
42 en binario:      0000 0000 0000 0000 0000 0000 0010 1010
42 << 8:            0000 0000 0000 0000 0010 1010 0000 0000    (10752)
                                     ^^^^^^^^
                                     8 ceros = .0

0.5 en punto fijo:  0000 0000 0000 0000 0000 0000 1000 0000    (128)
                                     ^
                                     bit 7 = 0.5 (1/2)
```

$42.5 = 42 \ll 8 + 128 = 10752 + 128 = 10880$

Verificación: $10880 / 256 = 42.5$ 

El cajón en RAM para el número 42.5:

```
STACK:
cajon 1040: [ _fPvalue = 10880 ]
            → valor real = 10880 / 256 = 42.5
```

18. El factor de escala — por qué 256

El factor de escala es $2^{\text{bits}} = 2^8 = 256$. Es el número que conecta el valor interno con el valor real:

```
valor_real = _fPvalue / 256
_fPvalue   = valor_real * 256
```

Algunos ejemplos para que quede claro:

```
_fPvalue = 256   → 256 / 256 = 1.0
_fPvalue = 512   → 512 / 256 = 2.0
_fPvalue = 128   → 128 / 256 = 0.5
_fPvalue = 768   → 768 / 256 = 3.0
_fPvalue = 384   → 384 / 256 = 1.5
_fPvalue = 1     → 1 / 256 = 0.00390625 ← el epsilon
_fPvalue = 0     → 0 / 256 = 0.0
_fPvalue = -256  → -256 / 256 = -1.0
```

19. El epsilon — el valor mínimo representable

Con 8 bits para la parte fraccionaria, el valor mínimo que puedes representar (distinto de cero) es:

```
epsilon = 1 / 2^8 = 1 / 256 = 0.00390625
```

Es el valor que corresponde a `_fPvalue = 1`. Es la “unidad” más pequeña del sistema.

El subject del ex02 lo llama con la letra griega ω (omega) pero es el mismo concepto. En matemáticas formales se llama epsilon.

La importancia del epsilon: en el ex02, los operadores `++` y `--` aumentan o disminuyen el valor en **exactamente un epsilon** — la cantidad mínima posible. Hacerlo es tan sencillo

como `_fPvalue += 1` o `_fPvalue -= 1`.

20. La precisión tiene límites — qué no puedes representar

No puedes representar cualquier decimal. La precisión es fija en 0.00390625. Los valores entre dos múltiplos del epsilon no son representables:

```
0.1 → _fPvalue = roundf(0.1 * 256) = roundf(25.6) = 26
26 / 256 = 0.1015625 ← no es exactamente 0.1
```

```
0.3 → _fPvalue = roundf(0.3 * 256) = roundf(76.8) = 77
77 / 256 = 0.30078125 ← no es exactamente 0.3
```

La diferencia respecto al float: el error de representación del punto fijo es **predecible y acotado**. Sabes exactamente cuánto puede desviarse (máximo $\epsilon/2 = 0.00195$). Con float, el error depende de las operaciones y puede acumularse de formas impredecibles.

PARTE 3 — CONVERTIR HACIA Y DESDE PUNTO FIJO

21. De int a punto fijo — el constructor int

```
Fixed::Fixed(const int n) {
    std::cout << "Int constructor called" << std::endl;
    _fPvalue = n << _bits;
}
```

Para representar el entero 42 en punto fijo:

```
Queremos que valor_real = 42
valor_real = _fPvalue / 256
→ _fPvalue = 42 * 256 = 10752
```

La operación `n << _bits` es `n << 8` que equivale a `n * 256`. Multiplicar por 256 es desplazar 8 bits a la izquierda.

Trazando en RAM con `Fixed b(10)` :

```
n = 10
_fPvalue = 10 << 8 = 10 * 256 = 2560

cajon de b: [ _fPvalue = 2560 ]
valor real: 2560 / 256 = 10.0 
```

22. El desplazamiento de bits — << y >> explicados desde cero

El desplazamiento de bits es un operador que mueve todos los bits de un número hacia la izquierda o la derecha.

Desplazamiento a la izquierda — <<

```
x << n    // equivale a x * 2^n
```

Cada posición que se desplaza a la izquierda dobla el valor:

```
5 en binario:    00000101
5 << 1:          00001010 = 10    (5 * 2)
5 << 2:          00010100 = 20    (5 * 4)
5 << 3:          00101000 = 40    (5 * 8)
5 << 8:          00000101 00000000 = 1280    (5 * 256)
```

Los bits que “salen” por la izquierda se pierden. Los nuevos bits que entran por la derecha son siempre cero.

Desplazamiento a la derecha — >>

```
x >> n    // equivale a x / 2^n (truncando la parte decimal)
```

Cada posición desplaza a la derecha divide por dos (truncando):

```
10752 en binario:  0000 0000 0000 0000 0010 1010 0000 0000
10752 >> 8:        0000 0000 0000 0000 0000 0000 0010 1010 = 42
                                     ^^^^^^^^
                                     los 8 bits fraccionarios se pierden — truncad
```

Por qué es más eficiente que multiplicar o dividir

El procesador tiene una instrucción dedicada para el desplazamiento de bits que tarda un

solo ciclo de reloj. La multiplicación y la división son operaciones más complejas. En sistemas de tiempo real (audio, gráficos), esta diferencia importa.

23. De float a punto fijo — el constructor float

```
Fixed::Fixed(const float f) {  
    std::cout << "Float constructor called" << std::endl;  
    _fPvalue = roundf(f * (1 << _bits));  
}
```

Para 42.42f :

```
Paso 1: (1 << _bits) = (1 << 8) = 256 ← el factor de escala  
Paso 2: f * 256 = 42.42 * 256 = 10859.52 ← escalar el float  
Paso 3: roundf(10859.52) = 10860 ← redondear al entero más cercano  
Paso 4: _fPvalue = 10860
```

```
Verificación: 10860 / 256 = 42.421875 ← lo más cerca posible de 42.42 con 8 bits
```

Por qué `(1 << _bits)` y no simplemente `256` : son matemáticamente equivalentes. Pero `(1 << _bits)` expresa la intención — el factor de escala se calcula a partir del número de bits. Si el diseño cambiara a `_bits = 16` , el factor se actualizaría solo.

24. roundf — por qué redondear y no truncar

`roundf` es la única función de `<cmath>` que el subject autoriza explícitamente. Está ahí por una razón.

Sin `roundf` , si simplemente castearas a `int`, truncarías:

```
(int)(10859.52) = 10859  
10859 / 256 = 42.41796875  
Error respecto a 42.42: 0.00203125
```

Con `roundf` , redondeas al entero más cercano:

```
roundf(10859.52) = 10860  
10860 / 256 = 42.421875  
Error respecto a 42.42: 0.001875 ← menor error
```

`roundf` da siempre la representación más precisa posible. El subject dice que el programa debe funcionar como en el ejemplo — si no usas `roundf` y hay diferencias de precisión, la salida no coincide.

25. `roundf` vs `floorf` vs `ceilf` — las diferencias

Función	Comportamiento	Ejemplo con 10859.52	Ejemplo con 10859.49
<code>roundf(x)</code>	Al entero más cercano (≥ 0.5 sube)	10860	10859
<code>floorf(x)</code>	Hacia abajo siempre	10859	10859
<code>ceilf(x)</code>	Hacia arriba siempre	10860	10860
<code>(int)(x)</code>	Trunca hacia cero	10859	10859

Para la conversión de float a punto fijo, `roundf` es la correcta — minimiza el error de representación.

Con números negativos:

```
roundf(-10859.52) = -10860 ← redondea al más cercano (más negativo)
floorf(-10859.52) = -10860 ← hacia abajo (más negativo)
ceilf(-10859.52)  = -10859 ← hacia arriba (menos negativo)
(int)(-10859.52)  = -10859 ← trunca hacia cero
```

26. De punto fijo a float — `toFloat`

```
float Fixed::toFloat(void) const {
    return (float)(_fPvalue) / (1 << _bits);
}
```

Es la operación inversa al constructor float:

```
_fPvalue = 10860
(float)(10860) / 256 = 42.421875f
```

27. El cast en C++ — (float), (int)

Un cast es una conversión explícita de un tipo a otro. La sintaxis en C++98:

```
(float)_fPvalue    // convierte _fPvalue (int) a float
(int)3.14f         // convierte 3.14f (float) a int — trunca a 3
```

En C++ moderno existen casts más explícitos:

```
static_cast<float>(_fPvalue)    // equivalente — más verboso
```

En 42 con C++98 ambos son válidos. El (float) clásico es más conciso.

28. Por qué sin el cast la división trunca

```
// SIN cast — ambos operandos son int:
_fPvalue / (1 << _bits)
10860 / 256 = 42    // división entera — trunca la parte decimal
                  // .421875 desaparece

// CON cast — un operando es float:
(float)(_fPvalue) / (1 << _bits)
42.421875f / 256 ... wait, (1 << _bits) es int pero...
```

En C++, cuando uno de los dos operandos de una división es `float`, el otro se convierte automáticamente a `float` y la división es flotante:

```
(float)(10860) / 256
// (float)(10860) = 42.421875f (conversión a float)
// 256 se convierte automáticamente a 256.0f
// 42.421875f / 256.0f = 0.16571... wait no
```

Vamos a trazar correctamente:

```
(float)(_fPvalue) / (1 << _bits)
= (float)(10860) / (1 << 8)
= 42.421875 no espera...
```

Corrección: el cast convierte `_fPvalue` a `float` antes de la división:

```
(float)(10860) = 10860.0f    ← ahora es float
(1 << 8) = 256               ← sigue siendo int
10860.0f / 256               ← int se promueve a float
= 42.421875f 
```

La regla: si cualquiera de los dos operandos es `float`, la operación entera se convierte en flotante. Basta con castear uno de los dos.

29. De punto fijo a int — `toInt`

```
int Fixed::toInt(void) const {
    return _fPvalue >> _bits;
}
```

Divide el valor interno entre 256 descartando la parte fraccionaria:

```
_fPvalue = 10860
10860 >> 8 = 10860 / 256 = 42 (la parte .421875 se pierde — trunca)
```

También puedes implementarlo con división entera — son equivalentes:

```
return _fPvalue / (1 << _bits);    // también correcto
```

`toInt` siempre trunca — no redondea. `42.9` se convierte en `42`, no en `43`.

PARTE 4 — QUÉ ES LA SOBRECARGA DE OPERADORES

30. Ad-hoc polymorphism — el mismo símbolo, comportamientos distintos

La sobrecarga de operadores es dar nuevo significado a un operador existente cuando se aplica a tipos definidos por el usuario.

Ya lo usas sin saberlo desde el ex00:


```
int a = 5 + 3;                // + suma enteros
float b = 1.5f + 2.5f;       // + suma floats
std::string c = "Hola" + " mundo"; // + concatena strings
```

El símbolo `+` hace tres cosas distintas según los tipos. Eso es **ad-hoc polymorphism** — el mismo operador, comportamientos distintos dependiendo del contexto.

Tú defines qué hace `+` cuando los operandos son `Fixed`:

```
Fixed a(5.0f), b(2.5f);
Fixed c = a + b; // tú defines esto — sin sobrecarga sería error de compilación
```

31. Cómo declara el compilador un operador sobrecargado

Un operador sobrecargado es simplemente una función con un nombre especial. El nombre es `operator` seguido del símbolo:

```
operator+   operator-   operator*   operator/
operator>   operator<   operator>=   operator<=   operator==   operator!=
operator++  operator--  operator=   operator<<
```

El compilador traduce la sintaxis de operador a llamada de función:

```
a + b      → a.operator+(b)      o operator+(a, b)
a == b     → a.operator==(b)
++a        → a.operator++()
a++        → a.operator++(0)     ← el compilador pasa 0 automáticamente
cout << fixed → operator<<(cout, fixed)
```

32. Operador como método vs operador como función libre

Algunos operadores pueden implementarse de las dos formas. La diferencia está en quién es el operando izquierdo.

Como método de clase — el objeto actual es el operando izquierdo:

```
a + b → a.operator+(b)
// a es el objeto sobre el que se llama el método (this)
// b es el parámetro
```

Como función libre — ambos operandos son parámetros:

```
cout << fixed → operator<<(cout, fixed)
// cout es el primer parámetro
// fixed es el segundo parámetro
```

`operator<<` DEBE ser función libre porque el operando izquierdo es `ostream`, no `Fixed`. Si fuera método de `Fixed`, `Fixed` estaría a la izquierda:

```
// método — Fixed tendría que estar a la izquierda:
fixed << cout // esto no tiene sentido
```

Todos los operadores del ex02 excepto `operator<<` son métodos de `Fixed`.

PARTE 5 — LOS OPERADORES DE COMPARACIÓN

33. La firma de los operadores de comparación

Los seis operadores de comparación tienen todos la misma estructura:

```
// en Fixed.hpp — dentro de la clase
bool operator>(const Fixed &other) const;
bool operator<(const Fixed &other) const;
bool operator>=(const Fixed &other) const;
bool operator<=(const Fixed &other) const;
bool operator==(const Fixed &other) const;
bool operator!=(const Fixed &other) const;
```

Desglosando `bool operator>(const Fixed &other) const`:

- `bool` — devuelve verdadero o falso
- `operator>` — es el operador mayor que
- `(const Fixed &other)` — recibe una referencia constante al otro operando
- `const` al final — este método no modifica ningún atributo

El `const` al final es una promesa al compilador: “ejecutar esta comparación no cambia el

estado del objeto". Eso permite llamar a estos operadores sobre objetos `const` :

```
const Fixed a(5.0f), b(2.5f);  
if (a > b) ...    // funciona porque operator> es const
```

34. Por qué operar sobre `_value` y no sobre `toFloat`

Podrías comparar convirtiendo a float:

```
bool Fixed::operator>(const Fixed &other) const {  
    return toFloat() > other.toFloat();    // funciona pero es peor  
}
```

Esto funciona, pero tiene dos problemas:

Problema 1 — imprecisión:

La conversión a float puede perder precisión. Dos `Fixed` con valores `_fPvalue` distintos pero muy cercanos podrían dar el mismo float después de la conversión, haciendo que `a == b` devuelva `true` cuando `_fPvalue` de `a` y `b` son distintos.

Problema 2 — ineficiencia:

Hacer dos conversiones a float por comparación tiene un coste innecesario.

Comparar `_fPvalue` directamente es exacto y eficiente:

```
bool Fixed::operator>(const Fixed &other) const {  
    return _fPvalue > other._fPvalue;    // exacto, rápido, sin conversiones  
}
```

La relación de orden entre `_fPvalue` es idéntica a la relación de orden entre los valores reales que representan. Si `_fPvalue` de `A` es mayor que `_fPvalue` de `B`, entonces `A` representa un número mayor que `B`. Sin excepciones.

Puedes acceder a `other._fPvalue` directamente aunque sea privado — estás dentro de la clase `Fixed`. Las restricciones de acceso se aplican entre clases distintas, no dentro de la misma clase.

35. El `if` con `return` — las tres formas equivalentes

Los tres estilos producen exactamente el mismo resultado:

```
// Forma 1 — con if/else
bool Fixed::operator>(const Fixed &other) const {
    if (_fPvalue > other._fPvalue)
        return true;
    else
        return false;
}

// Forma 2 — sin else (el return del if ya sale de la función)
bool Fixed::operator>(const Fixed &other) const {
    if (_fPvalue > other._fPvalue)
        return true;
    return false;
}

// Forma 3 — directa (la más idiomática en C++)
bool Fixed::operator>(const Fixed &other) const {
    return _fPvalue > other._fPvalue;
}
```

Una comparación (`>`, `<`, `==` ...) en C++ ya devuelve `true` o `false`. El `if` y el `return true/false` son redundantes. La forma 3 es la estándar en C++.

PARTE 6 — LOS OPERADORES ARITMÉTICOS

36. La firma — por qué devuelven `Fixed` por valor

```
Fixed operator+(const Fixed &other) const;
Fixed operator-(const Fixed &other) const;
Fixed operator*(const Fixed &other) const;
Fixed operator/(const Fixed &other) const;
```

Devuelven `Fixed` por valor — sin `&`. ¿Por qué no por referencia?

Porque crean un **objeto nuevo**. El resultado de `a + b` no es `a` ni es `b` — es un tercer objeto que nació de la operación. Si devolvieras una referencia, ¿a qué apuntaría?

```
Fixed &Fixed::operator+(const Fixed &other) const {
```

```

    Fixed resultado(...);    // variable local
    return resultado;        // PELIGRO — resultado muere al salir de la función
                             // quien llame a + tendría una referencia a memoria li
                             // DANGLING REFERENCE
}

```

No puedes devolver referencia a una variable local. El único camino es devolver por valor — una copia del resultado.

```

Fixed Fixed::operator+(const Fixed &other) const {
    return Fixed(toFloat() + other.toFloat());
    // crea un Fixed temporal con el resultado, lo copia al llamador, muere
}

```

37. El objeto temporal — qué es y cuándo nace y muere

En el ex01, la línea:

```
a = Fixed(1234.4321f);
```

Crea un **objeto temporal** anónimo — un `Fixed` sin nombre que vive solo durante esa expresión.

Trazando:

1. `Fixed(1234.4321f)` → Float constructor called
Se crea en el stack un `Fixed` sin nombre
`_fPvalue = roundf(1234.4321 * 256) = roundf(315950.6...) = 315951`
2. `a = (ese Fixed sin nombre)`
`a.operator=(temporal)` → Copy assignment operator called
`a._fPvalue = 315951`
3. El punto y coma termina la expresión
El objeto temporal ya no se necesita
Destructor called ← el temporal muere aquí

Por eso el output del ex01 tiene "Float constructor called", "Copy assignment operator called" y "Destructor called" seguidos — son el nacimiento, uso y muerte del temporal.

Los temporales también aparecen en el ex02:

```
Fixed const b(Fixed(5.05f) * Fixed(2));
// Fixed(5.05f) → temporal 1
// Fixed(2)      → temporal 2
// operator*     → temporal 3 (el resultado)
// Fixed(temporal 3) → b se construye desde el temporal 3
// temporales 1, 2, 3 mueren
```

38. Aritmética con toFloat vs aritmética directa sobre _value

Hay dos formas de implementar los operadores aritméticos:

Forma A — con toFloat (la más simple):

```
Fixed Fixed::operator+(const Fixed &other) const {
    return Fixed(toFloat() + other.toFloat());
}
```

Convierte ambos a float, suma, vuelve a convertir. Simple pero introduce el error de representación flotante en cada operación.

Forma B — directa sobre _value (más precisa para + y -):

Para suma y resta, los `_value` tienen el mismo factor de escala — se pueden sumar directamente:

```
Fixed Fixed::operator+(const Fixed &other) const {
    Fixed result;
    result.setRawBits(_fPvalue + other._fPvalue);
    return result;
}
```

Trazando:

```
a._fPvalue = 10 * 256 = 2560    (representa 10.0)
b._fPvalue = 5 * 256 = 1280    (representa 5.0)
resultado._fPvalue = 2560 + 1280 = 3840
3840 / 256 = 15.0 
```

Para multiplicación y división la forma directa requiere ajuste de escala — es más compleja. La forma A con `toFloat` es más simple y suficiente para CPP02.

PARTE 7 — LOS OPERADORES DE INCREMENTO Y DECREMENTO

39. Pre-incremento ++a — firma, comportamiento, traza en RAM

```
// declaración en Fixed.hpp
Fixed &operator++();

// implementación en Fixed.cpp
Fixed &Fixed::operator++() {
    _fPvalue += 1;
    return *this;
}
```

Lo que hace paso a paso:

1. Incrementa `_fPvalue` en 1
2. Devuelve una referencia al objeto ya modificado

Trazando ++a con `a._fPvalue = 0`:

```
ANTES:
    cajon de a: [ _fPvalue = 0 ]

operator++() se ejecuta:
    _fPvalue += 1 → cajon de a: [ _fPvalue = 1 ]
    return *this  → referencia a a

DESPUÉS:
    cajon de a: [ _fPvalue = 1 ]
    valor real: 1 / 256 = 0.00390625
```

Por qué devuelve `Fixed &` y no `Fixed`:

Devuelve el propio objeto modificado. No hay objeto nuevo. Una referencia es eficiente — no hay copia. Además, en C++ el pre-incremento nativo devuelve referencia, y la sobrecarga debe comportarse igual.

Uso:

```
Fixed a;          // a._fPvalue = 0
Fixed b = ++a;    // a se incrementa PRIMERO, luego b = a (ya incrementado)
                  // a._fPvalue = 1, b._fPvalue = 1
```

40. Post-incremento a++ — firma, comportamiento, traza en RAM

```
// declaración en Fixed.hpp — el int sin nombre es el truco
Fixed operator++(int);

// implementación en Fixed.cpp
Fixed Fixed::operator++(int) {
    Fixed temp = *this;    // guarda copia del estado actual
    _fPvalue += 1;         // incrementa el original
    return temp;           // devuelve la copia (estado ANTES del incremento)
}
```

Lo que hace paso a paso:

1. Guarda una copia del estado actual en `temp`
2. Incrementa `_fPvalue` en 1
3. Devuelve `temp` (el estado **antes** del incremento)

Trazando `a++` con `a._fPvalue = 1`:

```
ANTES:
    cajon de a: [ _fPvalue = 1 ]

operator++(int) se ejecuta:
    Fixed temp = *this
    → temp._fPvalue = 1    (copia del estado actual)

    _fPvalue += 1
    → cajon de a: [ _fPvalue = 2 ]

    return temp
    → devuelve temp (con _fPvalue = 1 — el estado anterior)
    temp muere (era una variable local)
```

DESPUÉS:


```
cajon de a: [ _fPvalue = 2 ] ← incrementado
valor devuelto: Fixed con _fPvalue = 1 ← el estado anterior
```

Por qué devuelve `Fixed` por valor y no `Fixed &`:

Devuelve `temp` — una variable local. Una vez que la función termina, `temp` muere. No puedes devolver referencia a una variable local — sería una dangling reference. Debes devolver por valor — se hace una copia de `temp` antes de que muera.

Uso:

```
Fixed a;          // a._fPvalue = 0
Fixed b = a++;    // b = a (ANTES del incremento), luego a se incrementa
                  // b._fPvalue = 0, a._fPvalue = 1
```

La consecuencia de rendimiento:

El post-incremento crea una copia temporal. El pre-incremento no. Por eso en C++ es costumbre preferir `++i` sobre `i++` cuando el valor devuelto no importa — el pre-incremento es más eficiente. Para tipos simples como `int` el compilador lo optimiza, pero para tipos complejos (como `Fixed`) la diferencia es real.

41. El parámetro `int dummy` — por qué existe

Pre y post incremento tienen una diferencia crucial en la firma:

```
Fixed &operator++();    // pre-incremento — sin parámetro
Fixed operator++(int);  // post-incremento — con parámetro int sin nombre
```

El `int` del post-incremento **no tiene nombre y nunca se usa**. Su único propósito es que el compilador pueda distinguir entre pre y post incremento, que de otra forma tendrían exactamente la misma firma.

El compilador genera automáticamente la llamada correcta:

```
++a;    // compilador llama a operator++()      — sin argumento
a++;    // compilador llama a operator++(int)    — pasa 0 automáticamente
```

Tú nunca pasas ese `int` manualmente. Es completamente transparente.

La misma convención aplica al decremento:

```
Fixed &operator--();    // pre-decremento
Fixed  operator--(int); // post-decremento
```

42. Pre-decremento --a

```
Fixed &Fixed::operator--() {
    _fPvalue -= 1;
    return *this;
}
```

Idéntico al pre-incremento pero restando 1. Decrementa y devuelve el objeto ya modificado.

43. Post-decremento a--

```
Fixed Fixed::operator--(int) {
    Fixed temp = *this;
    _fPvalue -= 1;
    return temp;
}
```

Idéntico al post-incremento pero restando 1. Guarda el estado, decrementa, devuelve el estado anterior.

44. El epsilon en contexto — qué significa _value += 1

El subject dice:

"operators, which will increase or decrease the fixed-point value by the smallest representable ω (omega), such that $1 + \omega > 1$ "

`_fPvalue += 1` añade 1 al valor entero interno. ¿Cuánto representa eso en el número real?

```
valor real antes:  0 / 256 = 0.0
_fPvalue += 1
valor real después: 1 / 256 = 0.00390625
```

cambio = 0.00390625 = epsilon 

La salida del main del ex02 lo confirma:

```
Fixed a;           // a = 0.0  (_fPvalue = 0)
cout << a;         // 0
cout << ++a;       // 0.00390625  ← +1 epsilon
cout << a;         // 0.00390625  ← sigue siendo el mismo (pre-incremento modificado)
cout << a++;       // 0.00390625  ← devuelve el estado antes del incremento
cout << a;         // 0.0078125   ← 2 * epsilon (dos incrementos totales)
```

¿Por qué `_fPvalue += 1` y no `_fPvalue += (1 << _bits)` (que sería `+= 256`)?

Porque `1 << 8 = 256` representaría añadir `1.0` completo. Añadir 1 al entero interno añade exactamente la unidad mínima — el epsilon.

PARTE 8 — EL OPERADOR << DE INSERCIÓN

45. Por qué `operator<<` no puede ser método de la clase

Si sobrecargas `<<` como método de `Fixed`:

```
class Fixed {
public:
    std::ostream &operator<<(std::ostream &out) const;
};
```

El compilador lo traduce así:

```
fixed << cout
// equivale a:
fixed.operator<<(cout)
```

Pero tú quieres `cout << fixed` — el stream a la izquierda y el `Fixed` a la derecha. Con método, el orden sería al revés.

La solución: función libre, donde ambos operandos son parámetros:

```
// FUERA de la clase
std::ostream &operator<<(std::ostream &out, const Fixed &fixed) {
    out << fixed.toFloat();
    return out;
}
```

```
}
```

Así el compilador traduce `cout << fixed` como `operator<<(cout, fixed)` — el orden correcto.

46. La firma completa desglosada símbolo por símbolo

```
std::ostream &operator<<(std::ostream &out, const Fixed &fixed)
```

- `std::ostream &` — tipo de retorno: referencia al stream de salida
- `operator<<` — nombre de la función (el operador de inserción)
- `std::ostream &out` — el stream de salida (cout, fichero...), por referencia porque lo vas a modificar, no const porque escribes en él
- `const Fixed &fixed` — el Fixed que vas a imprimir, por referencia para no copiar, const porque solo lees

Por qué `std::ostream &` como tipo de retorno:

Para permitir el encadenamiento de `<<`:

```
std::cout << a << " y " << b << std::endl;
// se procesa de izquierda a derecha:
// operator<<(cout, a)      → devuelve referencia a cout
// operator<<(cout, " y ")  → devuelve referencia a cout
// operator<<(cout, b)      → devuelve referencia a cout
// operator<<(cout, endl)   → devuelve referencia a cout
```

Si devolviera `void`, la segunda `<<` no tendría stream al que escribir.

Si devolviera `ostream` por valor, copiaría el stream — `ostream` no es copiable, error de compilación.

47. El encadenamiento — por qué devuelve `ostream&`

```
std::ostream &operator<<(std::ostream &out, const Fixed &fixed) {
    out << fixed.toFloat();
    return out;    // ← devuelves el mismo stream que recibiste
```

```
}
```

`return out` devuelve el mismo stream que recibiste — no creas nada nuevo. Es la referencia al objeto `cout` que ya existía antes. Quien llame a `<<` recibe ese mismo `cout` y puede seguir usándolo.

PARTE 9 — LAS FUNCIONES ESTÁTICAS MIN Y MAX

48. Por qué son `static` — y qué significa `static` en un método

Un método normal se llama sobre un objeto:

```
a.announce();    // announce se ejecuta con this = &a
```

El método sabe qué objeto es el "actual" gracias a `this`.

Un método `static` no pertenece a ningún objeto concreto — pertenece a la clase en general:

```
Fixed::min(a, b);    // se llama con el nombre de la clase, no con un objeto  
                    // no hay this dentro de min
```

`min` y `max` no necesitan `this` — comparan dos objetos que se pasan como parámetros. No tiene sentido que "pertenezcan" a un objeto concreto. Son funciones de la clase `Fixed` en general.

Si no fueran `static`, tendrías que llamarlas de esta forma rara:

```
a.min(a, b);    // ¿por qué min "pertenece" a a? no tiene sentido semántico
```

Con `static`:

```
Fixed::min(a, b);    // min pertenece a Fixed como clase — tiene sentido
```

Un método `static` no puede:

- Usar `this`
- Acceder a atributos no estáticos directamente

- Llamar a métodos no estáticos directamente

Un método static puede:

- Acceder a atributos estáticos
 - Llamar a otros métodos estáticos
 - Recibir objetos como parámetros y operar sobre ellos
-

49. La propagación de const — por qué hay dos versiones

```
static Fixed      &min(Fixed &a, Fixed &b);           // versión no const
static const Fixed &min(const Fixed &a, const Fixed &b); // versión const
```

¿Por qué dos versiones? Por la **propagación de const**.

En C++, cuando recibes un objeto `const`, no puedes devolver una referencia no constante a él — estarías “quitando” el `const`:

```
const Fixed a(5.0f), b(2.5f);

// Versión no const:
Fixed &m = Fixed::min(a, b);
// ERROR — a y b son const, no puedes devolver Fixed & (no const) a uno de ellos
// sería como prometer que el resultado es modificable cuando en realidad no lo es

// Versión const:
const Fixed &m = Fixed::min(a, b);
// OK — devuelves const Fixed &, mantienes la promesa de que no se modificará
```

Si solo tuvieras la versión no const:

```
const Fixed a(5.0f), b(2.5f);
Fixed::min(a, b); // ERROR — los parámetros no son const Fixed &, son Fixed &
                  // no puedes pasar objetos const a parámetros no const
```

El compilador elige automáticamente cuál versión usar según los argumentos:

```
Fixed a(5.0f), b(2.5f);
Fixed::min(a, b);           // llama a la versión no const

const Fixed c(5.0f), d(2.5f);
```

```
Fixed::min(c, d); // llama a la versión const
```

50. Devolver referencia — por qué no por valor

```
Fixed &Fixed::min(Fixed &a, Fixed &b) {  
    if (a.getRawBits() < b.getRawBits())  
        return a;  
    return b;  
}
```

Devuelve una **referencia** al menor de los dos — no una copia. ¿Por qué?

Eficiencia: no crear una copia innecesaria — `a` y `b` ya existen, devolver referencia a uno de ellos es una sola instrucción.

Permite modificar el resultado: si devuelves referencia, puedes hacer cosas como:

```
Fixed a(5.0f), b(2.5f);  
Fixed::min(a, b) = Fixed(1.0f); // modifica b (el menor) directamente
```

¿Puedes devolver referencia a `a` o `b` sin problemas? Sí — `a` y `b` son parámetros por referencia que apuntan a los objetos originales. Los objetos originales viven fuera de `min`. Devolver referencia a ellos es seguro — no son variables locales.

51. Cómo llamarlas desde el main

```
Fixed a(5.0f), b(2.5f);  
  
std::cout << Fixed::min(a, b) << std::endl; // imprime 2.5  
std::cout << Fixed::max(a, b) << std::endl; // imprime 5  
  
// el resultado es una referencia al objeto menor/mayor  
// el operator<< lo recibe y llama a toFloat() internamente
```

El main del ex02 tiene:

```
std::cout << Fixed::max(a, b) << std::endl;
```

`Fixed::max(a, b)` devuelve una referencia al mayor. El `operator<<` recibe esa referencia

como `const Fixed &fixed` y llama a `fixed.toFloat()` para imprimir.

PARTE 10 — EX03: BSP Y LA CLASE POINT

52. Qué pide el ex03

El ex03 pide implementar una función que determine si un punto está **estrictamente dentro** de un triángulo o no.

Si el punto está en un vértice o en una arista del triángulo, la función devuelve `false`. Solo devuelve `true` si está en el interior.

Para esto necesitas:

1. La clase `Point` — representa un punto 2D con coordenadas `Fixed const x` e `Fixed const y`
2. La función `bsp` — implementa el algoritmo de detección

BSP significa **Binary Space Partitioning** — una técnica usada en motores de videojuegos para organizar el espacio 3D. En el ex03 es una versión simplificada en 2D.

53. La clase Point en OCF

```
class Point {
    private:
        Fixed const _x;    // CONSTANTE — no se puede modificar tras la construcci
        Fixed const _y;    // CONSTANTE — no se puede modificar tras la construcci

    public:
        Point();            // constructor por defecto
        Point(const float x, const float y); // constructor con coordenadas
        Point(const Point &other);           // constructor de copia
        Point &operator=(const Point &other); // operador de asignación
        ~Point();                        // destructor

        Fixed getX(void) const;
        Fixed getY(void) const;
};
```

54. El problema de los atributos const — Fixed const x

Los atributos `Fixed const _x` e `Fixed const _y` son **constantes de instancia**. Una vez que el objeto se construye, sus coordenadas no pueden cambiar nunca.

Esto genera dos consecuencias importantes:

Consecuencia 1 — deben inicializarse en la lista de inicialización

```
// MAL — ERROR de compilación
Point::Point(const float x, const float y) {
    _x = Fixed(x);    // ERROR — no puedes asignar a un atributo const en el cuerpo
    _y = Fixed(y);    // ERROR — mismo problema
}

// BIEN — lista de inicialización
Point::Point(const float x, const float y) : _x(x), _y(y) {
    // _x y _y se construyen con los valores correctos directamente
}
```

Cuando el cuerpo del constructor empieza a ejecutarse, el objeto ya existe en RAM con todos sus atributos contruidos. Para un `const`, eso significa que ya tiene su valor final — no puede cambiar. La única oportunidad de darle valor es en la lista de inicialización, antes de que el objeto exista.

Trazando en RAM con `Point p(3.5f, 2.0f)`:

```
Lista de inicialización:
_x se construye con valor 3.5f → Fixed(3.5f) = _x._fPvalue = roundf(3.5*256)
_y se construye con valor 2.0f → Fixed(2.0f) = _y._fPvalue = 2*256 = 512

Cuerpo { }:
vacío — nada que hacer
_x ya tiene 896, _y ya tiene 512
intentar _x = algo → ERROR de compilación
```

Consecuencia 2 — el constructor por defecto debe inicializar las constantes

```
// MAL — ERROR de compilación
Point::Point() {
    // _x y _y no se inicializan → pero son const → deben tener valor → ERROR
}
```

```
// BIEN
Point::Point() : _x(0), _y(0) {
    // _x = Fixed(0), _y = Fixed(0)
}
```

55. El operator= con atributos const — la trampa

La OCF obliga a tener `operator=`. Pero el `operator=` intenta copiar los atributos de `other` en `*this`. Los atributos `const` **no se pueden modificar tras la construcción**.

¿Contradicción?

Sí. Esta es la trampa del ex03.

La solución común en 42: implementar el `operator=` sin hacer nada (o casi nada):

```
Point &Point::operator=(const Point &other) {
    (void)other;    // suprimir el warning de parámetro no usado
    return *this;
}
```

Esto satisface la OCF (el operador existe) pero no copia nada — porque no puede. Un `Point` una vez creado no cambia sus coordenadas.

Alternativa — generar un warning en compilación si se intenta usar:

```
Point &Point::operator=(const Point &other) {
    if (this != &other) {
        // No se puede hacer nada — los atributos son const
        // Esto es normal para objetos con atributos const
    }
    return *this;
}
```

El corrector de 42 acepta ambas formas. Lo importante es que el operador exista y compile.

56. El algoritmo BSP — cómo saber si un punto está en un triángulo

Dado un triángulo con vértices A, B, C y un punto P, ¿cómo saber si P está dentro?

El método más eficiente usa el **producto vectorial** (cross product). La idea:

Para cada arista del triángulo ($A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$), el punto P debe estar **al mismo lado** de todas las aristas para estar dentro del triángulo.

Si P está al mismo lado de las tres aristas $\rightarrow$ dentro. Si está al lado opuesto de alguna $\rightarrow$ fuera. Si está exactamente sobre una arista $\rightarrow$ el producto es cero $\rightarrow$ devuelve false (el subject pide esto).

57. El producto vectorial — el corazón del algoritmo

El producto vectorial en 2D de dos vectores (ax, ay) y (bx, by) es:

$$(ax, ay) \times (bx, by) = ax * by - ay * bx$$

El signo del resultado indica de qué lado está un punto respecto a una línea:

- Positivo $\rightarrow$ a la izquierda de la línea
- Negativo $\rightarrow$ a la derecha de la línea
- Cero $\rightarrow$ sobre la línea

Para cada arista del triángulo, calculamos el "signo" de P respecto a esa arista:

```
Para la arista A-B:
vector arista: (B.x - A.x, B.y - A.y)
vector al punto: (P.x - A.x, P.y - A.y)
signo = (B.x - A.x) * (P.y - A.y) - (B.y - A.y) * (P.x - A.x)
```

Si los tres signos son todos positivos o todos negativos $\rightarrow$ P está dentro. Si alguno es cero $\rightarrow$ P está sobre una arista $\rightarrow$ false. Si hay signos distintos $\rightarrow$ P está fuera $\rightarrow$ false.

58. Implementación completa del bsp

```
// bsp.cpp
#include "Point.hpp"

// Calcula el "signo" del producto vectorial
// Devuelve positivo, negativo, o cero
static Fixed crossProduct(Point const a, Point const b, Point const p) {
```

```

        return (b.getX() - a.getX()) * (p.getY() - a.getY())
            - (b.getY() - a.getY()) * (p.getX() - a.getX());
    }

    bool bsp(Point const a, Point const b, Point const c, Point const point) {
        Fixed d1 = crossProduct(a, b, point);
        Fixed d2 = crossProduct(b, c, point);
        Fixed d3 = crossProduct(c, a, point);

        // Si algún producto es exactamente cero → el punto está en una arista → fals
        Fixed zero(0);
        if (d1 == zero || d2 == zero || d3 == zero)
            return false;

        // Si todos tienen el mismo signo → el punto está dentro → true
        bool allPositive = (d1 > zero) && (d2 > zero) && (d3 > zero);
        bool allNegative = (d1 < zero) && (d2 < zero) && (d3 < zero);

        return allPositive || allNegative;
    }

```

Trazando con triángulo A(0,0), B(4,0), C(0,4) y punto P(1,1):

```

d1 = crossProduct(A, B, P)
    = (4-0) * (1-0) - (0-0) * (1-0)
    = 4 * 1 - 0 * 1
    = 4    (positivo)

d2 = crossProduct(B, C, P)
    = (0-4) * (1-0) - (4-0) * (1-0)
    = -4 * 1 - 4 * 1
    = -8    (negativo) ← wait, esto indicaría que está fuera...

```

Hmm, depende del orden de los vértices. Si los vértices están en orden antihorario, todos los signos serán positivos para puntos interiores. Si están en orden horario, todos serán negativos. El algoritmo funciona en ambos casos porque comprueba "todos positivos OR todos negativos".

Trazando con A(0,0), B(0,4), C(4,0) (orden diferente) y P(1,1):

```

d1 = crossProduct(A, B, P)
    = (0-0) * (1-0) - (4-0) * (1-0)
    = 0 - 4 = -4 (negativo)

d2 = crossProduct(B, C, P)

```

$$\begin{aligned}
 &= (4-0) * (1-4) - (0-4) * (1-0) \\
 &= 4*(-3) - (-4)*1 \\
 &= -12 + 4 = -8 \text{ (negativo)}
 \end{aligned}$$

```

d3 = crossProduct(C, A, P)
    = (0-4) * (1-0) - (0-0) * (1-4)
    = -4*1 - 0
    = -4 (negativo)

```

Todos negativos → P está dentro 

El algoritmo funciona independientemente del orden de los vértices — detecta tanto “todos positivos” como “todos negativos”.

PARTE 11 — LOS EJERCICIOS EXPLICADOS UNO A UNO

59. Ex00: My First Class in Orthodox Canonical Form

Concepto central: OCF básica y punto fijo sin conversiones.

Los archivos necesarios: Makefile, Fixed.hpp, Fixed.cpp, main.cpp

Lo que tiene la clase:

```

Privado:
    int _fPvalue          ← el valor interno
    static const int _bits = 8 ← los bits fraccionarios

Público:
    Fixed()                ← constructor por defecto: _fPvalue = 0
    Fixed(const Fixed &)   ← constructor de copia
    Fixed &operator=(const Fixed &) ← operador de asignación
    ~Fixed()               ← destructor
    int getRawBits() const ← getter — IMPRIME MENSAJE en ex00
    void setRawBits(int)   ← setter

```

La salida del main — trazada:

```

Fixed a;           → Default constructor called

```

```

Fixed b(a);      → Copy constructor called
                  getRawBits member function called (dentro del copy ctor)
                  Copy assignment operator called (si copy ctor usa *this =
Fixed c;          → Default constructor called
c = b;           → Copy assignment operator called
                  getRawBits member function called (dentro del operator=)
a.getRawBits()   → getRawBits member function called / 0
b.getRawBits()   → getRawBits member function called / 0
c.getRawBits()   → getRawBits member function called / 0
fin de scope     → Destructor called (c)
                  Destructor called (b)
                  Destructor called (a)

```

60. Ex01: Towards a more useful fixed-point number class

Concepto central: conversiones entre tipos y operator<<.

Lo que añade respecto al ex00:

Nuevos constructores:

```

Fixed(const int)    ← int a punto fijo con <<
Fixed(const float)  ← float a punto fijo con roundf

```

Nuevos métodos:

```

float toFloat() const ← punto fijo a float con cast y /
int toInt() const     ← punto fijo a int con >>

```

Nueva función libre:

```

ostream &operator<<(ostream &, const Fixed &) ← imprime toFloat()

```

Importante: en el ex01, `getRawBits` ya NO imprime el mensaje. Solo devuelve el valor.

El objeto temporal en el main:

```

a = Fixed(1234.4321f);
// 1. Float constructor called ← crea el temporal
// 2. Copy assignment operator called ← a = temporal
// 3. Destructor called         ← el temporal muere

```

61. Ex02: Now we're talking

Concepto central: sobrecarga completa de operadores — el punto fijo se comporta como un tipo nativo.

Lo que añade respecto al ex01:

```
6 comparadores: > < >= <= == !=
4 aritméticos:  + - * /
4 incrementos:  ++a  a++  --a  a--
4 static:       min(no const)  min(const)  max(no const)  max(const)
```

Sin mensajes de constructores/destructores en el output del main:

El subject dice "(constructor/destructor messages are removed in the example below)". Tu programa SÍ los imprime — el subject solo los quita para hacer más legible el ejemplo.

Verificación del main:

```
Fixed a;                                a._fPvalue = 0
Fixed const b(Fixed(5.05f) * Fixed(2)):
    Fixed(5.05f)._fPvalue = roundf(5.05*256) = roundf(1292.8) = 1293
    Fixed(2)._fPvalue = 2*256 = 512
    operator*: Fixed(5.05078125f * 2.0f) = Fixed(10.1015625f)
    b._fPvalue = roundf(10.1015625*256) = roundf(2586) = 2586

cout << a          → 0
cout << ++a        → _fPvalue=1 → 1/256 = 0.00390625
cout << a          → 0.00390625
cout << a++        → devuelve temp (fPvalue=1) → 0.00390625, a._fPvalue pasa a 2
cout << a          → 2/256 = 0.0078125
cout << b          → 2586/256 = 10.1015625 → cout muestra 10.1016
Fixed::max(a,b)    → b > a → devuelve b → 10.1016
```

62. Ex03: BSP (bonus)

Concepto central: uso de Fixed en un problema geométrico real.

Los archivos necesarios:

- Fixed.hpp / Fixed.cpp (del ex02)
- Point.hpp / Point.cpp (clase nueva)
- bsp.cpp (función libre)

- main.cpp (tus propios tests)

Casos de prueba que debes verificar:

```
// Triángulo con vértices A(0,0), B(4,0), C(0,4)
```

```
bsp(A, B, C, Point(1, 1))    → true    (punto interior)
bsp(A, B, C, Point(2, 2))    → false   (sobre la arista B-C: 2+2=4)
bsp(A, B, C, Point(0, 0))    → false   (vértice A)
bsp(A, B, C, Point(4, 0))    → false   (vértice B)
bsp(A, B, C, Point(2, 0))    → false   (sobre la arista A-B)
bsp(A, B, C, Point(5, 5))    → false   (fuera del triángulo)
bsp(A, B, C, Point(0.5, 0.5)) → true    (punto interior pequeño)
```

GLOSARIO COMPLETO

Término	Definición
OCF	Orthodox Canonical Form — las 4 funciones obligatorias desde CPP02
Constructor por defecto	Sin parámetros — crea el objeto en estado inicial válido
Constructor de copia	Crea un objeto nuevo a partir de otro existente — Fixed(const Fixed &)
Operador de asignación	Actualiza un objeto ya existente — Fixed &operator=(const Fixed &)
Destructor	Se llama automáticamente al destruir el objeto
Regla de los Tres	Si defines cualquiera de {destructor, copy ctor, copy assign}, necesitas los tres
Shallow copy	Copia byte a byte — peligrosa con punteros
Deep copy	Copia también los datos apuntados — lo que implementas manualmente
this	Puntero al objeto actual — tipo Fixed * — dirección del objeto en RAM

<code>*this</code>	El objeto actual desreferenciado — tipo <code>Fixed</code>
<code>return *this</code>	Devuelve referencia al objeto actual — permite encadenar <code>a = b = c</code>
Autoasignación	<code>a = a</code> — <code>this == &other</code> — la comprobación <code>if (this != &other)</code> la detecta
Lista de inicialización	<code>:</code> atributo(valor) antes del cuerpo — obligatoria para referencias y <code>const</code>
<code>static</code> (atributo)	Existe una sola vez para toda la clase — compartido por todos los objetos
<code>static</code> (método)	Se llama sin objeto — <code>Fixed::min()</code> — no tiene <code>this</code>
<code>const</code> (método)	El método no modifica ningún atributo — permite llamarlo sobre objetos <code>const</code>
<code>const</code> (atributo)	No puede cambiar tras la construcción — debe inicializarse en lista
Punto fijo	Representación de decimales con enteros y un número fijo de bits fraccionarios
<code>_bits = 8</code>	Número de bits fraccionarios — determina la precisión
Factor de escala	$2^8 = 256$ — multiplicas para ir a punto fijo, divides para salir
Epsilon (ω)	Valor mínimo representable = $1/256 = 0.00390625$ — corresponde a <code>_fPvalue = 1</code>
<code>_fPvalue << 8</code>	Conversión de <code>int</code> a punto fijo — multiplica por 256
<code>_fPvalue >> 8</code>	Conversión de punto fijo a <code>int</code> — divide truncando por 256
<code>roundf(x)</code>	Redondea <code>x</code> al <code>float</code> entero más cercano — de <code>cmath</code>
<code>floorf(x)</code>	Redondea hacia abajo siempre
<code>ceilf(x)</code>	Redondea hacia arriba siempre
Cast (<code>float</code>)	Conversión explícita de tipo — <code>(float)x</code>
División entera	<code>int / int</code> trunca la parte decimal
División flotante	<code>float / float</code> mantiene decimales
Ad-hoc polymorphism	El mismo operador hace cosas distintas según los tipos

Sobrecarga de operadores	Dar nuevo significado a un operador para tipos propios
operator como método	El objeto actual (this) es el operando izquierdo
operator como función libre	Ambos operandos son parámetros — necesario para operator<<
Objeto temporal	Fixed sin nombre que vive solo durante una expresión
Dangling reference	Referencia a variable local que ya murió — crash garantizado
Pre-incremento ++a	Incrementa y devuelve Fixed& al objeto ya modificado
Post-incremento a++	Guarda copia, incrementa, devuelve la copia — parámetro int dummy
int dummy	El int sin nombre del post-incremento — existe solo para distinguir la firma
Propagación de const	Si los parámetros son const, el retorno también debe ser const
operator<<	Función libre — ostream& a la izquierda, Fixed a la derecha
ostream& como retorno	Permite encadenar cout << a << b << c
BSP	Binary Space Partitioning — algoritmo para determinar posición en el espacio
Producto vectorial 2D	$(axby - aybx)$ — su signo indica de qué lado está un punto respecto a una línea
Punto interior	Mismo signo en los tres productos vectoriales del triángulo
Punto en arista	Un producto vectorial es exactamente cero